

**Software de Gestión Administración Comunidades y Gastos Comunes
(DAS) Documento Arquitectura de Software
Versión 2.0**

Identificación de Documento

Identificación	DAS SGGC Administración de Comunidades y Gastos Comunes
Proyecto	Sistema de Gestión de Administración de Comunidades y Gastos Comunes
Versión	2.0

Documento mantenido por	Grupo 1: Pamela Acuña Gabriel Martínez Daniel Castro
Fecha de última revisión	05-04-2026
Fecha de próxima revisión	12-04-2026

Documento aprobado por	
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
25-03-2026	0.1	Creación de la estructura base y redacción de Cap. 1.	Pamela Acuña
28-03-2026	0.5	Incorporación de Requerimientos Funcionales y No Funcionales.	Gabriel Martínez
03-04-2026	1.0	Corrección de la Arquitectura Modular en capas y versión final para entrega.	Daniel Castro

ÍNDICE

1. INTRODUCCIÓN	1
1.1 Contexto del problema	1
1.2. Propósito	1
1.3. Ámbito	2
1.4. Definiciones, acrónimos y abreviaciones	3
1.5. Resumen ejecutivo	3
1.6. Arquitectura del sistema	4
2. VISIÓN DEL SISTEMA	4
2.1. Descripción general del sistema	4
2.2. Objetivos del sistema	5
2.3. Requerimientos funcionales	6
2.4. Requerimientos no funcionales	10
2.5. Supuestos y dependencias	12
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	12
3.1. Estilo arquitectónico Adoptado	12
3.2. Estilo de arquitectura según contexto del sistema	13
3.3. Patrón de arquitectura aplicado	13
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	14
4.1 VISTA DE ESCENARIO	14
4.1.1 Propósito	14
4.1.2 Actores, son quienes usarán el sistema	14
4.1.3 Diagrama general de casos de uso	15
4.1.4 Diagrama de casos de uso específicos	17
4.1.5 Lista de casos de uso	18
4.1.6. Especificación de Casos de Uso	20
4.2. VISTA LÓGICA	25
4.2.1 Propósito	25
4.2.2 Diagrama de clases	25
4.2.3 Descripción Diagrama de Clases	26
4.3 . VISTA DE IMPLEMENTACIÓN / DESARROLLO	26
4.3.1 Propósito	26
4.3.2 Diagrama de Componente	27
4.3.3 Descripción diagrama de Componente	27
4.3.4 Diagrama de paquete	29
4.3.5 Descripción diagrama de paquete	29
4.4. VISTA DE PROCESOS	32
4.4.1 Propósito	32
4.4.2 Diagrama de Actividad	32

4.4.3 Descripción Diagrama de Actividad	33
4.5. VISTA FÍSICA	33
4.5.1 Propósito	33
4.5.2 Diagrama de despliegue	33
4.5.3 Descripción Diagrama de Despliegue	33
5. REQUISITOS DE CALIDAD	34
5.1 Propósito	34
5.2 Atributos de calidad	34
5.3. Reglas y criterios de evaluación de calidad.	34
6. PRINCIPIOS DE DISEÑO APLICADOS	34
6.1 Propósito	34
6.2 Principios de diseño	34
7. PROTOTIPO	35
7.1 Propósito	35
7.2 Mockup	35
7.3 Justificar herramientas de prototipado	35
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	35
6.1. Propósito	35
6.2. Lista de verificación	35
6.3. Análisis y métricas de resultados	35
9. CONTROL DE VERSIONES	35
7.1. Propósito	35
7.2. Control de versión utilizado (justificar el tipo de control de versión utilizad (fecha, semántica o secuencial)	35
7.3. Justificar herramientas de versionamiento	35
10. CONCLUSIONES	35
11. BIBLIOGRAFÍA	35

1. INTRODUCCIÓN

1.1 Contexto del problema

Actualmente en muchas comunidades se presenta una problemática que se repite una y otra vez sin importar si la comunidad consiste en un edificio, conjunto de edificios, condominios o conjuntos residenciales, que es la complejidad y poca eficiencia en la gestión administrativa y de gastos comunes.

Si bien hasta hace algunos años se operaba de manera íntegramente empírica y manual en la gestión diaria, hoy los proyectos de construcción son cada día más grandes e integrales por lo que el aumento de recursos a administrar y considerar son mucho mayores surgiendo así un aumento notable de complejidad.

Actualmente las unidades o proyectos son a mayor escala, consideran variadas instalaciones de uso común y muchos más metros cuadrados que administrar.

Sin la presencia de digitalización y automatización es muy difícil ser eficiente y preciso a nivel operativo y de negocio.

Consecuencias de esto se evidencian importantes problemas para quienes deben administrar como;

Inestabilidad Financiera: Existe un deficiente control del flujo de caja, lo que impacta directamente en la liquidez y dificulta el cumplimiento oportuno de las obligaciones financieras del edificio.

Fallas Operativas: La gestión de cobros resulta ineficiente y se evidencian retrasos constantes en la ejecución de las mantenciones programadas de los espacios comunes.

Crisis de Confianza y Transparencia: La dependencia exclusiva de procesos de cálculo y registro manuales ha incrementado el margen de error. Esto se ha traducido en un alza significativa de los reclamos, mermando gravemente la confianza de los residentes (tanto propietarios como arrendatarios) hacia la administración.

Ausencia de Canales Transaccionales Autónomos: Adicionalmente, las comunidades carecen de métodos de pago en línea integrados. Esto obliga a los residentes a depender exclusivamente de transferencias bancarias manuales o dinero en efectivo, lo que incrementa los retrasos en las rendiciones, introduce fricción operativa e incrementa la posibilidad de errores humanos durante la conciliación bancaria que debe realizar el administrador.

1.2. Propósito

El propósito del presente documento es definir, estructurar y comunicar la arquitectura de software para el nuevo **Sistema de Administración de Comunidades y Gastos Comunes** en el desarrollo de una solución de software configurable para edificios y condominios que permita administrar propiedades, usuarios vinculados, cálculo y cobros de gastos comunes, morosidad, comunicaciones, solicitudes e informes, mejorando la eficiencia administrativa, la transparencia y la trazabilidad de la gestión comunitaria.

Mediante la implementación de este sistema y la definición de su arquitectura, se busca erradicar los errores operativos derivados de la digitación manual y proveer un motor transaccional dual. Este motor habilitará un canal de pago automatizado e inmediato para el residente vía portal web, manteniendo a la vez un flujo administrativo controlado para registrar pagos de contingencia (efectivo/transferencia), optimizando los tiempos de gestión y restaurando la transparencia financiera de la comunidad.

1.3. Ámbito

El ámbito de este proyecto comprende el diseño, desarrollo e implementación del **Sistema de Administración de Comunidades y de Gastos Comunes**. El sistema centralizará la información y automatizará los flujos de trabajo administrativos.

El sistema debe poder ser utilizado por:

- comunidades de edificios
- condominios
- conjuntos residenciales con múltiples propiedades

El sistema debe soportar que una propiedad pueda estar asociada a uno o varios actores según su situación:

- propietario
- inmobiliaria dueña del inmueble
- arrendatario
- corredor de propiedades

También debe considerar que una notificación pueda ser enviada a múltiples destinatarios relacionados con una misma propiedad o comunidad.

1.3.1. Alcance del Sistema (Dentro del Ámbito)

- **Módulo de Gestión de Usuarios y Propiedades:** Registro y mantención (CRUD) de la información de copropietarios, arrendatarios, personal administrativo y el maestro de departamentos (incluyendo la superficie en M2 de cada unidad).
- **Módulo de Finanzas y Prorratio:** Motor de cálculo automatizado para la distribución de gastos comunes y extraordinarios, aplicando reglas de negocio basadas en el prorratio por metro cuadrado.
- **Módulo de Pagos y Consolidación Híbrida:** Procesamiento automatizado de pagos en línea a través de una pasarela de pagos externa integrada en el Portal del Residente (Webpay/Transbank) con actualización instantánea de saldos. Asimismo, provee

herramientas en el Portal Administrativo para forzar la actualización y 'cambio de estado' de deudas cuando el residente efectúe pagos extra-portal (efectivo o transferencias directas), emitiendo de igual forma comprobantes digitales.

- **Módulo de Comunicación:** Canal digital para la emisión de notificaciones automáticas de cobro y una bandeja de entrada para la trazabilidad de solicitudes o reclamos de los residentes.

1.3.2. Exclusiones (Fuera del Ámbito)

Para garantizar la viabilidad y el enfoque de esta primera versión del sistema, se excluyen explícitamente las siguientes funcionalidades:

- **Gestión de Recursos Humanos:** No se incluirá un módulo para el cálculo de remuneraciones, pago de cotizaciones o control de asistencia del personal del edificio (conserjes, aseo).
- **Control de Acceso e IoT:** El sistema no interactúa con hardware del edificio, como barreras de estacionamiento, cámaras de seguridad o controles de acceso biométrico.

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
DAS	Documento de Arquitectura de Software. Es el presente documento, que proporciona una visión arquitectónica global del sistema.
SGGC	Sistema de Gestión de Gastos Comunes. Nombre asignado a la solución de software.
CRUD	<i>Create, Read, Update, Delete</i> (Crear, Leer, Actualizar, Eliminar). Acrónimo técnico que hace referencia a las cuatro operaciones básicas de gestión de datos en el sistema.
MVC	<i>Model-View-Controller</i> (Modelo-Vista-Controlador). Patrón de diseño de software que separa la aplicación en tres componentes principales para facilitar su desarrollo y mantenimiento.
Pasarela de Pagos (API)	Servicio externo (ej. Transbank/Webpay) encargado de procesar de forma segura y sincrónica las transacciones financieras con entidades bancarias.

1.5. Resumen ejecutivo

El presente Documento de Arquitectura de Software (DAS) describe la solución tecnológica propuesta para resolver deficiencias administrativas y operativas en la administración de comunidades. Ante la ineficiencia, falta de transparencia y los errores derivados de la actual gestión manual, se propone el desarrollo del **Sistema de Gestión de Administración de Comunidades y Gastos Comunes**. Esta

plataforma digital está orientada a automatizar y transparentar el ciclo financiero completo del recinto habitacional.

A nivel técnico, el sistema ha sido diseñado bajo una Arquitectura Modular en capas complementada con integración a servicios externos. Esta arquitectura permite organizar el sistema en componentes funcionales claramente separados, estructurando una plataforma web bifronte preparada para interactuar con proveedores externos como servicios de correo electrónico (SMTP) y pasarelas de pago (ej. Webpay, Transbank), manteniendo canales alternativos de registro manual para mitigar escenarios de contingencia operativa.

Este enfoque evita la complejidad de arquitecturas distribuidas como microservicios, pero permite escalabilidad funcional, flexibilidad y mantenibilidad en el tiempo.

La implementación del Sistema aportará un valor inmediato al negocio al automatizar el cálculo de prorrateso exacto basado en metros cuadrados (M2), centralizar el registro de pagos y establecer un control proactivo de la morosidad. En consecuencia, esta transformación digital permitirá estabilizar el flujo de caja, erradicar los errores de digitación y, como beneficio principal, restaurar la confianza de la comunidad de propietarios y arrendatarios mediante el acceso a información financiera clara, auditable y en tiempo real.

1.6. Arquitectura del sistema

El sistema adopta una arquitectura modular en capas, en la cual la lógica se organiza en componentes internos y módulos de integración.

Se contemplan los siguientes elementos:

- **Cientes Web (Interfaces Bifrontes):** Portal de Autogestión del Residente y Módulo de Control del Administrador.
- **Servidor de Aplicación (Backend):** Lógica de negocio distribuida en capas (Controlador, Servicio, Repositorio).
- **Base de Datos Relacional:** Almacenamiento e integridad de datos transaccionales y de auditoría.
- **Servicios Externos de Integración:** Servicio de correo (notificaciones asincrónicas) y Pasarela de pagos externa (procesamiento bancario sincrónico).

El sistema se comunica con estos servicios externos mediante APIs, permitiendo desacoplar la lógica interna de proveedores externos.

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El Sistema de Gestión de Administración de Comunidades y Gastos Comunes es una solución tecnológica web e integrada, diseñada bajo un enfoque arquitectónico bifronte. Por un lado, provee un **Módulo Administrativo** robusto para la gestión operativa y crear perfiles, cálculo prorratesado automatizado y control financiero de la copropiedad. Por otro lado, ofrece un **Portal del Residente** de autogestión, adaptado para los diferentes actores vinculados a las unidades (Propietarios, Arrendatarios, Corredores de Propiedades e Inmobiliarias). El sistema centraliza la información en una base de datos relacional y se conecta con servicios externos para resolver de extremo a extremo el flujo transaccional de cobros, notificaciones y recaudación de fondos, ya sea mediante canales

digitales automatizados o registros manuales de caja. En conjunto, el Sistema transforma un ecosistema administrativo propenso al error y la desinformación en una operación automatizada, auditable y transparente.

2.2. Objetivos del sistema

2.2.1. Objetivo General

Desarrollar e implementar una plataforma de software web configurable y escalable que optimice la administración operativa y financiera de condominios y edificios, garantizando la trazabilidad absoluta de los flujos de dinero (pagos en línea y manuales), facilitando la interacción de múltiples perfiles de usuario y mejorando la transparencia comunitaria mediante notificaciones automatizadas.

2.2.2. Objetivos Específicos

Para alcanzar el objetivo general, el sistema deberá cumplir con los siguientes objetivos específicos:

- Automatizar el Prorratio: Calcular de forma automática y exacta los gastos comunes mensuales de cada inmueble, utilizando como base matemática la superficie útil en metros cuadrados (m²) de cada unidad y sus complementos (estacionamientos, bodega), eliminando los errores de cálculo manual.
- Centralizar la Información: Crear un registro digital único y actualizado que vincule a los inmuebles con sus respectivos residentes (propietarios o arrendatarios) y su historial de pagos.
- Controlar y Reducir la Morosidad: Implementar rutinas de seguimiento que identifiquen automáticamente las cuentas impagas, calculan los recargos correspondientes y emitan notificaciones de cobro oportunas a los deudores.
- Habilitar la Autogestión y Transparencia: Proveer a los residentes de una interfaz web intuitiva donde puedan consultar su estado de cuenta en tiempo real, visualizar el desglose de los gastos del edificio y descargar sus comprobantes de pago.
- Optimizar los Tiempos Administrativos: Reducir significativamente las horas-hombre que la administración dedica actualmente a la digitación de datos, consolidación de planillas y atención de consultas rutinarias sobre saldos adeudados.
- Autonomía Transaccional: Integrar una pasarela de pagos externa (Webpay/Transbank) en el portal del residente para permitir la recaudación electrónica sincrónica y la actualización inmediata de estados financieros, minimizando la intervención humana.
- Contingencia Operativa: Proveer herramientas de conciliación manual y cambio de estado de deudas para el Administrador, permitiendo la recepción segura de pagos extra-portal (efectivo/transferencia) sin perder la integridad ni la auditoría del flujo de caja.
- Flexibilidad de Roles: Soportar un modelo de datos flexible capaz de asociar y diferenciar los permisos, visualizaciones y responsabilidades de copropietarios, arrendatarios, corredores de propiedades e inmobiliarias sobre una misma unidad residencial.

2.3. Requerimientos funcionales

A continuación, se detallan todas las funcionalidades que el sistema debe proveer para satisfacer los objetivos del negocio. Se han redactado sin ambigüedades, especificando acciones exactas para garantizar su correcta interpretación e implementación.

Módulo 1: Gestión de Usuarios y Propiedades

ID	Nombre de Funcionalidad	Descripción Específica y Reglas de Negocio
RF-01	Gestión de comunidades	El sistema debe permitir registrar y administrar una o más comunidades, indicando nombre, tipo de comunidad, dirección, reglamento interno, datos de contacto y estado.
RF-02	Gestión de torres, bloques y unidades	El sistema debe permitir registrar la estructura física de la comunidad, incluyendo torres, bloques, pisos, estacionamientos, bodegas y propiedades o unidades habitacionales.
RF-03	Gestión de propiedades	El sistema debe permitir crear, editar, consultar y desactivar propiedades, registrando su identificador, ubicación, superficie y estado de ocupación.
RF-04	Gestión de usuarios	El sistema debe permitir registrar, modificar y desactivar usuarios del sistema con información de identificación, contacto, rol, medio preferido de notificación y estado.
RF-05	Asociación de personas a una propiedad	El sistema debe permitir asociar una propiedad a uno o varios usuarios con distintos roles: propietario, inmobiliaria dueña del inmueble, arrendatario, corredor de propiedades, residente autorizado. Debe ser posible definir vigencia, prioridad de contacto y tipo de responsabilidad de cada vínculo.
RF-06	Gestión de roles y permisos	El sistema debe permitir asignar perfiles y permisos diferenciados según el tipo de usuario, asegurando acceso controlado a funcionalidades e información.
RF-07	Registro de ocupación de la propiedad	El sistema debe permitir registrar el estado de ocupación de una propiedad: habitada por propietario, arrendada, desocupada, en administración de corredor o en poder de inmobiliaria (con las fechas correspondientes al cambio de estado).
RF-08	Generación de gasto común	El sistema debe permitir generar el cálculo total del gasto común de un periodo adjuntando todos los gastos realizados por la administración.
RF-09	Generación de gasto común por propiedad	El sistema debe permitir generar cargos periódicos de gastos comunes por propiedad, considerando prorratio del cálculo total de gastos comunes generados en el periodo.
RF-10	Registro de pagos e integración híbrida	El sistema debe procesar la recaudación bajo dos modalidades fijas:1) Pago en línea (Residente): Procesamiento automático mediante la integración de una pasarela de pago externa (Webpay) en el portal, que actualiza el estado de la deuda a 'PAGADO' inmediatamente.2) Conciliación manual (Administrador): Permite al Administrador registrar pagos manuales recibidos fuera del portal (efectivo o transferencia bancaria directa) y forzar el 'cambio de estado' de la

		deuda de gasto común a 'PAGADO', asociando el ID del administrador operante para fines de auditoría.Reglas de Negocio: No se podrán registrar pagos con montos menores o iguales a cero (RN-08). Todo pago debe quedar asociado a una propiedad y a un período o documento de cobro (RN-09).
RF-11	Emisión de comprobantes	El sistema debe generar comprobantes o recibos por cada pago registrado, con opción de descarga y envío digital.
RF-12	Estado de cuenta por propiedad	El sistema debe permitir consultar el estado de cuenta histórico y vigente de cada propiedad, incluyendo cargos, pagos, deudas, intereses, multas y saldo total.
RF-13	Gestión de morosidad	El sistema debe identificar automáticamente propiedades morosas, clasificarlas por antigüedad de deuda, calcular intereses o recargos configurables y generar alertas de cobranza.
RF-14	Programación de notificaciones	El sistema debe permitir crear notificaciones automáticas o manuales asociadas a eventos como: emisión de gasto común, vencimiento próximo, atraso en el pago, pago registrado, multa aplicada, respuesta a solicitud, anuncio comunitario.
RF-15	Notificaciones múltiples	El sistema debe permitir enviar una misma notificación a más de un usuario relacionado con una propiedad o comunidad, por ejemplo propietario y arrendatario, o inmobiliaria y corredor.
RF-16	Configuración de destinatarios	El sistema debe permitir definir reglas de envío de notificaciones según tipo de evento, rol del usuario, estado de ocupación de la propiedad y prioridad de contacto.
RF-17	Historial de comunicaciones	El sistema debe mantener un historial de notificaciones enviadas, con fecha, canal, destinatarios, estado de entrega y contenido.
RF-18	Gestión de solicitudes	El sistema debe permitir que residentes o usuarios autorizados ingresen solicitudes de mantención, servicios, consultas o requerimientos administrativos.
RF-19	Gestión de reclamos y quejas	El sistema debe permitir registrar reclamos, derivarlos a responsables, asignar prioridad, controlar plazos y dejar evidencia de su resolución.
RF-20	Seguimiento de casos	El sistema debe permitir dar seguimiento al estado de solicitudes, incidentes y reclamos mediante estados como ingresado, en revisión, en proceso, resuelto y cerrado.
RF-21	Panel de control administrativo	El sistema debe ofrecer un panel con indicadores clave: total recaudado, deuda total, tasa de morosidad, pagos del período, solicitudes abiertas, notificaciones enviadas.
RF-22	Consultas e informes	El sistema debe generar informes sobre: propiedades, usuarios vinculados, estado de ocupación, pagos, deudas y morosidad, ingresos por período, solicitudes y reclamos, notificaciones emitidas. Este requerimiento se alinea directamente con lo solicitado en el documento base sobre generación de consultas e informes.

RF-23	Acceso de usuarios a información en tiempo real	El sistema debe permitir que administración, propietarios y residentes accedan en línea a la información relevante según su perfil, tal como plantea el caso.
RF-24	Búsqueda y filtros	El sistema debe permitir búsquedas avanzadas por propiedad, usuario, rol, estado de pago, mora, torre, bloque, período y tipo de incidencia.
RF-25	Auditoría de acciones	El sistema debe registrar acciones relevantes realizadas en la plataforma, incluyendo creación, modificación, eliminación, aprobación y envío de información.
RF-26	Parametrización del sistema	El sistema debe permitir configurar conceptos de cobro, reglas de cálculo, intereses por mora, plantillas de notificación, tipos de propiedades, tipos de usuarios y estructura organizacional de la comunidad.
RF-27	Exportación de información	El sistema debe permitir exportar reportes y consultas en formatos comunes como PDF y Excel.

2.4. Requerimientos no funcionales

Los siguientes requerimientos definen los atributos de calidad, restricciones técnicas y estándares de rendimiento que el sistema debe cumplir para garantizar una operación robusta y segura.

RNF-01	Usabilidad	El sistema debe ser intuitivo, fácil de usar y comprensible para usuarios administrativos y residentes con distintos niveles de alfabetización digital.
RNF-02	Disponibilidad	El sistema debe estar disponible al menos el 99% del tiempo mensual, excluyendo mantenciones programadas informadas previamente. En caso de indisponibilidad temporal de la pasarela de pagos externa (Webpay), el sistema debe mantenerse operativo permitiendo el flujo alternativo de conciliación y registro manual por parte del administrador para asegurar la continuidad operativa.
RNF-03	Rendimiento	El sistema debe responder las operaciones comunes de consulta en un tiempo no mayor a 3 segundos bajo carga normal.
RNF-04	Escalabilidad	El sistema debe poder adaptarse a comunidades pequeñas, medianas y grandes, soportando crecimiento en número de propiedades, usuarios, cobros y notificaciones.
RNF-05	Seguridad de acceso	El sistema debe exigir autenticación segura y control de acceso por roles y permisos.
RNF-06	Protección de datos	El sistema debe resguardar la confidencialidad e integridad de los datos mediante encriptación (HTTPS). Para las transacciones financieras a través del portal de residentes, el sistema no almacenará datos sensibles de tarjetas bancarias, delegando esta seguridad íntegramente a la pasarela de pagos externa, garantizando transacciones seguras de extremo a extremo.

RNF-07	Trazabilidad	El sistema debe mantener evidencia auditable de operaciones críticas como generación de cobros, pagos procesados por el portal, cambios de estado manuales ejecutados por el administrador (registrando su ID de usuario por seguridad), edición de datos y envío de notificaciones.
RNF-08	Confiabilidad	El sistema debe evitar pérdida de información ante fallos y garantizar consistencia en los datos transaccionales.
RNF-09	Respaldo y recuperación	El sistema debe realizar respaldos automáticos y permitir recuperación ante fallas o errores operativos.
RNF-10	Compatibilidad	El sistema debe ser accesible desde navegadores web modernos y dispositivos móviles.
RNF-11	Mantenibilidad	La solución debe estar construida de manera modular para facilitar correcciones, mejoras y nuevas adaptaciones para distintos tipos de comunidad.
RNF-12	Configurabilidad	El sistema debe permitir parametrizar reglas de negocio sin necesidad de modificar el código fuente para cada comunidad.
RNF-13	Interoperabilidad	El sistema debe poder integrarse con sistemas externos de pago, correo electrónico, mensajería y eventualmente ERP o sistemas contables.
RNF-14	Accesibilidad	La interfaz debe seguir criterios básicos de accesibilidad visual y de navegación para facilitar el uso por personas con distintas capacidades.
RNF-15	Notificaciones confiables	El sistema debe garantizar registro del resultado del envío de notificaciones y reintentos controlados en caso de falla.
RNF-16	Calidad de la información	El sistema debe validar datos obligatorios, formatos, duplicidades e inconsistencias antes de almacenar registros.
RNF-17	Soporte multiusuario	El sistema debe permitir el acceso concurrente de múltiples usuarios (administradores y residentes) de forma simultánea, sin sufrir degradación del rendimiento ni bloqueos en las transacciones de pago.
RNF-18	Localización y contexto normativo	El sistema debe poder adaptarse al idioma, moneda, formato de fecha y reglas de cobro definidas por cada administración o país.

2.5. Supuestos y dependencias

Para la correcta planificación, desarrollo e implementación del Sistema de Administración de Comunidades y Gastos Comunes, se han establecido las siguientes condiciones base. Cualquier alteración en estos puntos podría impactar el alcance, los plazos o la arquitectura del sistema.

2.5.1. Supuestos

- **Infraestructura del Usuario Final:** Se asume que tanto el personal administrativo como los residentes cuentan con dispositivos compatibles (computadoras, tablets o smartphones) y conexión estable a Internet para acceder a la plataforma web. El sistema no proveerá hardware.
- **Calidad y Disponibilidad de Datos Iniciales:** Se asume que, antes del paso a producción, la administración de cada comunidad entregará la "Data Maestra" depurada, estructurada y sin inconsistencias (ej. planilla Excel con el detalle exacto de los inmuebles, sus respectivos metros cuadrados (m2), datos de contacto de residentes actuales y saldos históricos). El sistema no se hará responsable por cálculos erróneos derivados de datos iniciales corruptos, ni por deudas previas que no hayan sido regularizadas en el saldo inicial mediante el flujo de conciliación correspondiente.
- **Alfabetización Digital:** Se asume que el público objetivo posee competencias digitales básicas para la navegación web estándar (ingreso de credenciales, llenado de formularios, descarga de PDFs).

2.5.2. Dependencias Externas

- **Infraestructura de Alojamiento (Cloud/Hosting):** El funcionamiento en ambiente de producción depende estrictamente de la contratación, configuración y disponibilidad de un servidor web (ej. VPS o servicio Cloud) y de un motor de Base de Datos Relacional para alojar la arquitectura en capas.
- **Servicio de Correos Transaccionales (SMTP):** Para dar cumplimiento al requerimiento funcional de notificaciones automáticas y alertas de cobro (RF-13), el sistema depende de la integración con un proveedor de servicios de correo electrónico externo (ej. SendGrid, Amazon SES o similar).
- **Pasarela de Pagos Externa (API):** La recaudación automatizada en línea a través del portal de residentes depende de la disponibilidad operativa, la vigencia de los certificados de seguridad y la correcta respuesta de la API del proveedor externo de servicios financieros (ej. Webpay/Transbank). En caso de indisponibilidad del proveedor, la arquitectura técnica derivará la operación de cobro exclusivamente hacia el flujo de dependencia interna del Módulo Administrativo (conciliación manual).
- **Aprobación del Comité de Administración:** El avance de las etapas críticas del desarrollo y el despliegue final dependen directamente de la disponibilidad del comité de administración para realizar las pruebas de aceptación de usuario (UAT) y validar que el motor de prorrateo funcione según sus reglas de negocio.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico Adoptado

El sistema propuesto para la gestión se basa en una arquitectura modular en capas, implementada como una aplicación web centralizada y preparada para integrarse con servicios externos mediante APIs. Esta arquitectura se compone de:

- **Capa de Presentación:** Interfaz web accesible por administradores y residentes. Para dar soporte al modelo transaccional del sistema, esta capa expone vistas bifrontes: el Portal del Residente (orientado a la consulta de estados de cuenta y pago en línea) y el Módulo de Control del Administrador (diseñado para la gestión operativa y la conciliación manual de

deudas).

- Capa de Lógica de Negocio: Contiene reglas de procesamiento como el cálculo de gastos comunes, control de morosidad, gestión de pagos y solicitudes. En esta capa se interceptan y ejecutan las reglas de validación financiera, impidiendo montos menores o iguales a cero e indexando cada flujo a una propiedad y período específico.
- Capa de Acceso a Datos: Encargada del manejo y la persistencia en la base de datos relacional. Garantiza la integridad referencial y almacena los registros de auditoría de todas las operaciones críticas.
- Capa de Integración: Permite la comunicación segura y desacoplada con servicios externos, específicamente con servicios de correo electrónico (SMTP) para el envío de notificaciones automáticas y pasarelas de pago (como la API de Webpay) para el procesamiento de transacciones financieras sincrónicas iniciadas por el residente.

Este enfoque permite mantener el sistema organizado de forma clara, separando responsabilidades, facilitando su desarrollo y preparándolo para futuras integraciones.

3.2. Estilo de arquitectura según contexto del sistema

La elección de una arquitectura modular en capas se justifica en base a las características del problema descrito en el caso:

- El sistema está enfocado en comunidades habitacionales, lo que representa un sistema de tamaño medio.
- Los procesos están bien definidos (registro de gastos, registro de pagos, morosidad, gestión de residentes).
- No se requiere alta escalabilidad distribuida (como microservicios), pero sí se necesita desacoplar la lógica interna de dependencias externas críticas (como el despacho de correos y la pasarela transaccional de Webpay).
- En caso de indisponibilidad temporal del proveedor transaccional en la capa de integración, el aislamiento del estilo en capas asegura la continuidad del negocio, permitiendo que la administración siga registrando la conciliación manual en la capa de negocio sin degradar la persistencia local.

Por ello, se adopta esta arquitectura modular con integración mediante APIs, lo que permite:

- Mantener simplicidad de desarrollo y facilitar el mantenimiento.
- Permitir la integración fluida con terceros (servicios SMTP y pasarelas financieras).
- Preparar el sistema para un crecimiento funcional futuro sin sobreingeniería.

3.3. Patrón de arquitectura aplicado

MVC (Modelo - Vista - Controlador) + patrón de Diseño Repository, Service y DTO.

- Modelo: entidades como Propietarios, Residentes, Departamentos, Pago, Solicitud. Representa el estado de los datos transaccionales de la copropiedad.
- Vista: interfaz web interactiva que se adapta dinámicamente según el perfil del usuario (desplegando el formulario de pago electrónico para el residente o la bitácora de conciliación de cajas para el administrador).
- Controlador: gestiona solicitudes del usuario. Actúa como el punto de entrada que recibe las peticiones HTTP del navegador (como la solicitud del administrador para cambiar manualmente el estado de una deuda) y las delega a los componentes lógicos

correspondientes.

Patrón Repository

- Encargado de acceder a la base de datos de manera directa y exclusiva.
- Permite desacoplar la lógica del almacenamiento. Contiene las operaciones CRUD y las consultas indexadas (como buscar deudas vigentes por propiedad o registrar la bitácora inalterable de auditoría).

Patrón Service

- Contiene la lógica de negocio pura y la orquestación de procesos:
 - cálculo de gastos comunes y prorrateos de copropiedad.
 - control de morosidad automático e intereses.
 - validaciones financieras críticas (verificar que los montos procesados sean mayores a cero y que el estado de la deuda mute a 'PAGADO' tras recibir la confirmación de la pasarela o la firma digital del administrador).

Patrones DTO (Data Transfer Object)

- Permite transferir datos eficientemente entre las capas del sistema sin exponer las entidades del modelo de manera directa.
- Mejora seguridad y control de información, evita mostrar toda la información interna. Por ejemplo, al comunicarse con la API de Webpay o al enviar comprobantes por correo electrónico, el DTO empaqueta estrictamente los campos financieros requeridos (monto, identificador de la unidad, período), protegiendo la confidencialidad del resto de los datos de la base de datos.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1 VISTA DE ESCENARIO

4.1.1 Propósito

La Vista de Escenario (o Vista de Casos de Uso) actúa como el elemento centralizador y validador de toda la arquitectura del software. Su propósito fundamental es abstraer las necesidades funcionales del negocio e ilustrar cómo los actores externos interactúan con las fronteras del sistema, asegurando que el diseño de software mitigue de raíz los problemas de opacidad financiera, errores de cálculo y fallas de comunicación identificados en el diagnóstico.

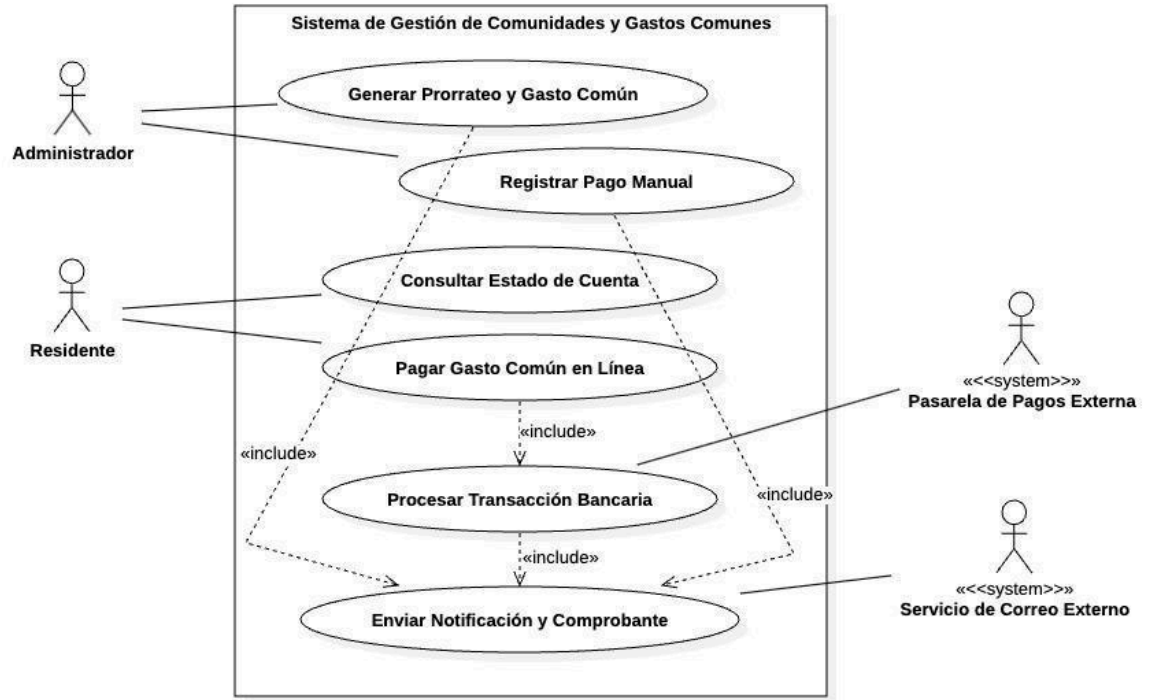
4.1.2 Actores del Sistema

Los actores representan roles externos o sistemas de terceros que interactúan directamente con la aplicación:

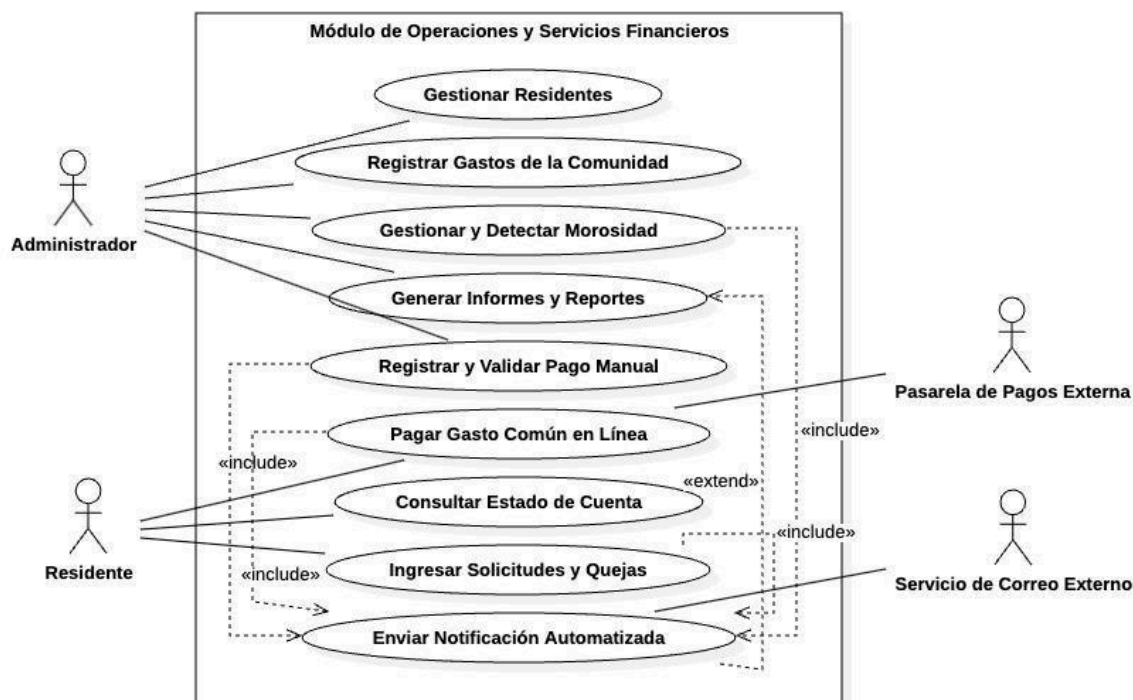
- **Administrador:** Actor humano (perfil interno). Es el encargado de la gestión operacional del condominio, el ingreso de gastos base, la generación automatizada del prorrateo y el registro manual de recaudaciones de contingencia (efectivo y transferencias directas).
- **Residente:** Actor humano (perfil externo). Rol genérico que consolida los permisos de interacción financiera y comunitaria para Co-propietarios, Arrendatarios, Inmobiliarias y Corredores de Propiedades vinculados a un inmueble.

- **Pasarela de Pagos Externa (API Webpay):** Sistema externo de tipo secundario. Procesa de manera sincrónica y segura los cobros electrónicos con validación bancaria inmediata.
- **Servicio de Correo Externo (API SMTP / SendGrid):** Sistema externo de tipo secundario. Se encarga del despacho masivo y asíncrono de colillas de cobro, comprobantes de pago y alertas de morosidad.

4.1.3 Diagrama general de casos de uso



4.1.4 Diagramas de Casos de Uso Específicos (Módulo Operacional y Financiero)



Propósito y Descripción:

El diagrama expuesto a continuación profundiza a nivel de sub-sistema en las interacciones críticas analizadas en las especificaciones (Sección 4.1.6). Su objetivo es modelar con rigurosidad técnica cómo las operaciones internas del Administrador (gestión de residentes, registro de gastos y control de morosidad) conviven y se integran con los flujos de autogestión del Residente (consultar estados de cuenta, enviar solicitudes y pagar en línea).

El diseño arquitectónico implementa dependencias explícitas hacia los actores secundarios (Pasarela de Pagos Externa y Servicio de Correo Externo), evidenciando que procesos automatizados como la detección de morosidad (CU-14) o el registro de pagos (CU-09) gatillan de forma obligatoria e integrada (<<include>>) las alertas perimetrales y de comunicación, mitigando de raíz los problemas de opacidad financiera y errores manuales diagnosticados en el origen del proyecto.

4.1.5 Lista de casos de uso

Gestión de comunidades y unidades

Código	Nombre	Actores
CU-01	Registrar comunidad residencial	Administrador
CU-02	Modificar comunidad residencial	Administrador
CU-03	Gestionar unidades residenciales	Administrador

Gestión de residentes

CU-04	Registrar residente	Administrador
CU-05	Modificar residente	Administrador
CU-06	Eliminar residente	Administrador
CU-07	Asociar residente a unidad residencial	Administrador
CU-08	Consultar información de residente	Administrador

Gestión de pagos

CU-09	Registrar pago manual	Administrador
CU-10	Pagar gasto común en línea	Residente, Pasarela de Pagos
CU-11	Generar comprobante de pago	Proceso Automático, Servicio de Correo Externo
CU-12	Consultar historial de pagos	Administrador/ Residente

Gestión de estado de cuenta

CU-13	Consultar estado de cuenta	Residente/Administrador
CU-14	Visualizar deuda actual	Residente

Gestión de morosidad

CU-15	Detectar residentes morosos	Proceso Automático
CU-16	Calcular deuda e intereses	Proceso Automático
CU-17	Generar aviso de cobro	Proceso Automático, Servicio de Correo Externo
CU-18	Consultar lista de morosos	Administrador

Gestión de solicitudes y quejas

CU-19	Registrar solicitud	Residente / Conserje
-------	---------------------	----------------------

CU-20	Registrar queja	Residente
CU-21	Asignar solicitud	Administrador
CU-22	Atender solicitud	Personal de mantención
CU-23	Cerrar solicitud	Administrador
CU-24	Consultar estado de solicitud	Residente

Gestión de informes

CU-25	Generar informe de pagos	Administrador
CU-26	Generar informe de morosidad	Administrador
CU-27	Generar reportes generales	Administrador

Autenticación y acceso

CU-28	Iniciar sesión	Usuario
CU-29	Cerrar sesión	Usuario

4.1.6. Especificación de Casos de Uso

A continuación, se presentan las especificaciones detalladas de los casos de uso más relevantes del Sistema de Gestión de Comunidades Residenciales. Estas especificaciones permiten describir de manera estructurada el comportamiento del sistema, considerando actores, condiciones, flujos principales y alternativos.

1. Registrar Pago Manual

Caso de Uso	Registrar Pago Manual	Identificador: CU-09
Actores	Administrador (Primario)	
Tipo	Primario	
Referencias		
Precondición	El residente y la unidad residencial deben estar previamente registrados en el sistema.	

Postcondición	El pago queda persistido en el sistema, se actualiza el saldo de la unidad residencial y se gatilla la generación del comprobante.
Descripción	Permite al Administrador registrar en forma manual un pago recibido en efectivo, transferencia directa o cheque para cubrir gastos comunes u otros conceptos.
Resumen	

Curso Normal:

N°	Ejecutor	Paso o Actividad
1	Administrador	Ingresa al sistema con sus credenciales de administración.
2	Administrador	Selecciona la opción “Registrar pago manual” en el módulo financiero.
3	Administrador	Busca y selecciona la unidad residencial correspondiente.
4	Administrador	Ingresa el monto, fecha, medio de pago y el número de documento de respaldo.
5	Sistema	Valida que los datos ingresados sean coherentes y obligatorios (campos no vacíos).
6	Sistema	Registra de forma persistente el pago en la base de datos.
7	Sistema	Actualiza el estado de cuenta y el saldo de la unidad residencial afectada.
8	Sistema	Invoca internamente al módulo de notificaciones para generar el comprobante de pago.

Curso Alternativo:

N°	Descripción
5A	[Datos Inválidos]: Si los datos ingresados son incorrectos, incompletos o el monto es menor o igual a cero, el sistema muestra un mensaje de error explicativo y solicita la corrección de los datos sin alterar la base de datos.
6A	[Fallo de Persistencia]: Si ocurre un error de infraestructura al guardar el registro, el sistema cancela la transacción completa (rollback), informa al usuario el fallo y no altera el saldo de la unidad.

2. Consultar estado de cuenta

Caso de Uso	Consultar Estado de Cuenta	Identificador: CU-12
Actores	Residente, Administrador (Primarios)	
Tipo	Primario	
Referencias		
Precondición	El usuario debe estar autenticado en el sistema y poseer un rol con permisos de lectura sobre la unidad residencial.	
Postcondición	El sistema despliega el estado financiero completo y actualizado de la unidad seleccionada.	
Descripción	Permite visualizar la deuda acumulada, los pagos validados con anterioridad, cobros del mes actual y el historial financiero general de la propiedad.	
Resumen		

Curso Normal:

N°	Ejecutor	Paso o Actividad
1	Usuario	Accede al menú principal de autogestión y selecciona "Estado de Cuenta".
2	Usuario	Selecciona el período (mes/año) específico que desea auditar.
3	Sistema	Consulta dinámicamente en la base de datos los saldos, cobros y abonos asociados a la unidad.
4	Sistema	Despliega en la interfaz el detalle consolidado de la deuda actual y el historial financiero.
5	Usuario	(Opcional) Hace clic en el botón de acción "Descargar Comprobante PDF".
6	Sistema	Renderiza el archivo en formato PDF y descarga el documento de manera local.

Curso Alternativo:

N°	Descripción
3A	[Sin Registros Financieros]: Si la unidad es nueva o no registra movimientos financieros en el período seleccionado, el sistema muestra un estado vacío con balance en \$0 e indica el mensaje "No se registran cobros ni pagos en este mes".
4A	[Error de Conexión]: Si la capa de acceso a datos no responde, el sistema bloquea temporalmente el panel, despliega una alerta técnica indicando "Servicio no disponible de momento" y registra la excepción en el log del backend.

3. Gestionar Morosidad (Proceso Automático)

Caso de Uso	Detectar Residentes Morosos	Identificador: CU-14
Actores	Proceso Automático (Primario), Administrador (Secundario)	
Tipo	Primario / Técnico	
Referencias		
Precondición	La fecha del sistema es posterior al día de vencimiento estipulado para el pago de los gastos comunes del mes en curso.	
Postcondición	Se actualiza el estado de las unidades deudoras a "Moroso", se aplican los recargos y se preparan las colas de notificación.	
Descripción	Rutina interna del servidor que identifica de forma automática a las unidades con deudas vencidas para recalcular saldos y emitir alertas automáticas.	
Resumen		

Curso Normal:

N°	Ejecutor	Paso o Actividad
1	Proceso Automático	Se ejecuta al cumplirse la hora y fecha límite programada en el sistema.
2	Proceso Automático	Examina los balances de todas las unidades residenciales registradas.
3	Proceso Automático	Identifica cuáles unidades mantienen saldos pendientes no

		cubiertos para el mes vencido.
4	Proceso Automático	Calcula los intereses y multas por atraso correspondientes según la regla de negocio.
5	Proceso Automático	Actualiza el estado de la unidad a "Moroso" e inyecta el nuevo saldo en la base de datos.
6	Proceso Automático	Genera los avisos de cobro y los deriva al Servicio de Correo Externo para su envío automático.
7	Administrador	Accede al panel de control y consulta la lista actualizada de morosos generada por el proceso.

Curso Alternativo:

N°	Descripción
3A	[Comunidad al Día]: Si al realizar el barrido todas las unidades se encuentran con sus saldos al día, el proceso termina la ejecución de forma limpia, guardando un registro en el log que indica: "Cierre de mes exitoso, cero morosidad detectada".

4. Registrar Solicitud o Queja

Caso de Uso	Registrar Solicitud o Queja	Identificador: CU-18
Actores	Residente, Conserje (Primarios)	
Tipo	Primario	
Referencias		
Precondición	El usuario debe estar correctamente registrado y con una cuenta activa vinculada a la comunidad.	
Postcondición	La solicitud queda registrada en estado "Abierta" con un ID único y se genera un ticket de seguimiento técnico.	
Descripción	Permite a los residentes o al personal de conserjería ingresar requerimientos de mantenimiento, quejas por ruidos o sugerencias operativas hacia la administración.	
Resumen		

Curso Normal:

N°	Ejecutor	Paso o Actividad
1	Usuario	Ingresa a la plataforma y selecciona el módulo de "Solicitudes y Quejas".
2	Usuario	Hace clic en el comando "Crear Nueva Solicitud".
3	Usuario	Selecciona la categoría (Mantenimiento, Reclamo, Sugerencia) y digita la descripción del caso.
4	Sistema	Genera un identificador numérico correlativo (TicketID) y marca el estado inicial como "Pendiente".
5	Sistema	Registra la solicitud asociándola a la cuenta del usuario y de la unidad habitacional.
6	Sistema	Dispara un correo electrónico al Administrador informando el ingreso de un nuevo requerimiento.

Curso Alternativo:

N°	Descripción
3A	[Campos Obligatorios Incompletos]: Si el usuario intenta enviar la solicitud dejando el cuadro de descripción en blanco o sin seleccionar una categoría, el sistema detiene el flujo, resalta el campo en rojo y advierte que la descripción es obligatoria para procesar el reclamo.

5. Generar informe

Caso de Uso	Generar Informe	Identificador: CU-24
Actores	Administración	
Tipo	Administrador (Primario)	
Referencias		
Precondición	Deben existir datos transaccionales previos registrados en la base de datos para el rango de fechas solicitadas.	
Postcondición	Se genera un reporte consolidado estructurado descargable en formato digital (PDF / Excel).	

Descripción	Permite a la administración extraer datos agregados sobre recaudación, gastos operacionales o tasas de morosidad para la toma de decisiones y rendición de cuentas.
Resumen	

Curso Normal:

N°	Ejecutor	Paso o Actividad
1	Administrador	Ingresa al menú administrativo y entra a la sección "Módulo de Reportes".
2	Administrador	Selecciona el tipo de informe requerido (Pagos, Gastos, Morosidad o Reporte General).
3	Administrador	Define los parámetros de filtrado (Rango de fechas o Torre/Bloque específico).
4	Sistema	Ejecuta la consulta SQL parametrizada y procesa los datos agregando sumatorias y porcentajes.
5	Sistema	Genera el informe visual en pantalla y habilita las opciones de exportación de archivos.
6	Administrador	Selecciona la opción de descarga en su formato de preferencia para guardarlo localmente.

Curso Alternativo:

N°	Descripción
4A	[Ausencia de Datos]: Si no existen transacciones registradas dentro de los parámetros o rango de fechas seleccionados por el administrador, el sistema no compila ningún documento, interrumpe el flujo y muestra en la pantalla de la interfaz el mensaje: "No existen datos disponibles para el criterio seleccionado".

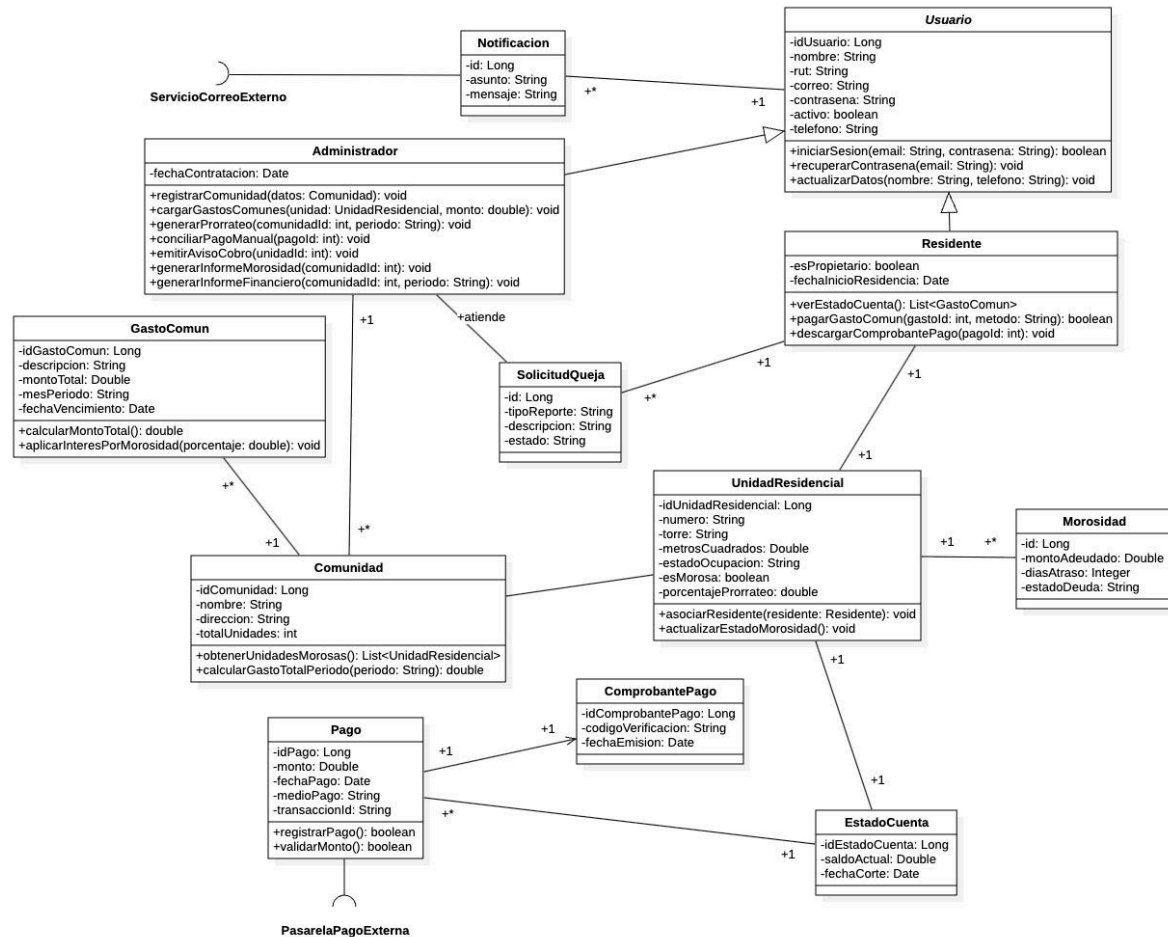
4.2. VISTA LÓGICA

4.2.1 Propósito

El propósito de la Vista Lógica es describir la estructura estática del Sistema de Gestión de Comunidades y Gastos Comunes a través de su modelo de objetos de negocio. A diferencia de las vistas funcionales, esta sección expone las abstracciones del dominio, sus encapsulamientos, tipos de

El diseño se centra en mitigar la opacidad operativa identificada en el diagnóstico del problema a través de un acoplamiento débil, utilizando el principio de herencia para la gestión de identidades y abstrayendo las dependencias hacia pasarelas bancarias y de mensajería mediante interfaces externas segregadas.

4.2.2 Diagrama de clases



4.2.3 Descripción Diagrama de Clases

El diagrama de clases modela conceptualmente las entidades principales del sistema, sus atributos obligatorios protegidos mediante encapsulamiento privado y la naturaleza de sus operaciones:

Estructura de Identidad (Herencia): La clase abstracta **Usuario** actúa como el núcleo de autenticación común, consolidando campos como identificadores únicos, nombres, correos electrónicos y contraseñas. De ella heredan mediante generalización las clases **Administrador** y **Residente**, separando los roles de control operacional de los de autogestión habitacional.

Modelo de Dominio Físico (Composición): La entidad **Comunidad** representa el conjunto condominal, la cual mantiene una relación de composición de uno a muchos (1 --- *) con **Propiedad** (Unidad Residencial). Esto garantiza que las propiedades dependan directamente de la existencia de la comunidad. Cada propiedad encapsula métricas como el número de unidad, piso y su porcentaje de prorrateo.

Asociación Residencial y Cobros: La clase **Residente** se vincula con una **Propiedad** para reflejar la relación de ocupación o dominio (cumpliendo con que una propiedad puede albergar múltiples usuarios). A su vez, cada **Propiedad** acumula el histórico de sus documentos de **GastoComun** emitidos para el cálculo de los períodos mensuales de cobro.

Estructura Transaccional: La clase **Pago** captura los eventos financieros del sistema (monto, fecha y forma de pago), manteniendo una relación asociativa directa con el **GastoComun** que liquida. Esta clase encapsula las reglas de negocio críticas, como la validación de montos mayores a cero y el estado de la conciliación (manual o por pasarela automática).

4.3. VISTA DE IMPLEMENTACIÓN / DESARROLLO

4.3.1 Propósito

La Vista de Implementación muestra cómo se organiza el código fuente del sistema, describiendo su estructura física en módulos, paquetes y componentes de software.

En el Sistema de Gestión de Comunidades y Gastos Comunes, esta vista se organiza bajo una arquitectura modular en capas (Layered Architecture), utilizando una jerarquía de directorios estándar basada en frameworks modernos (como Spring Boot). Esta organización permite una alta cohesión y un bajo acoplamiento, separando claramente las responsabilidades, facilitando el mantenimiento a largo plazo y preparando la infraestructura para la inyección de dependencias y la integración con servicios externos.

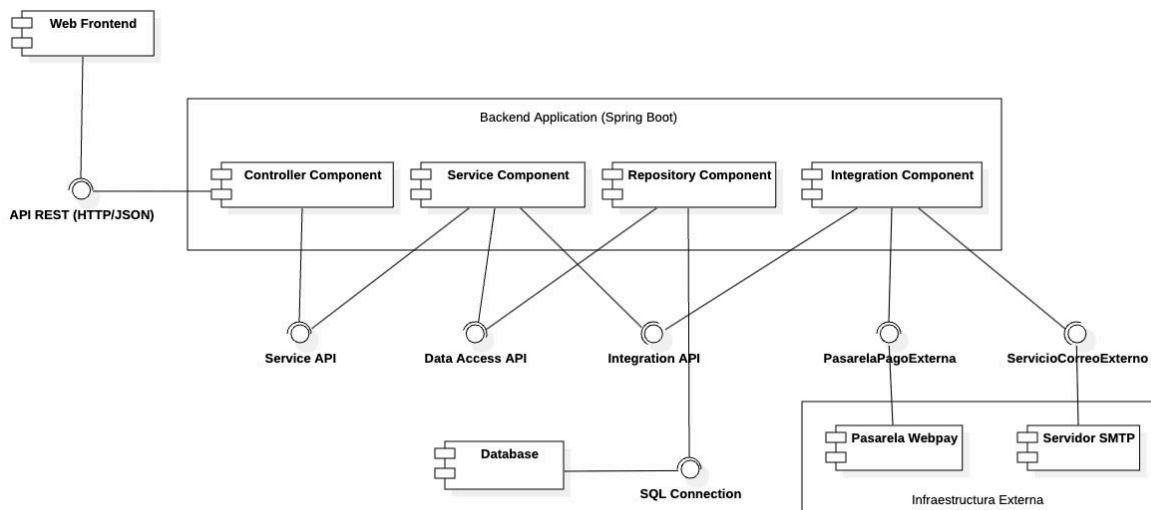
El sistema divide su código fuente en los siguientes paquetes principales:

- **controller**: Gestiona la capa de presentación (API REST). Expone los *endpoints*, recibe las peticiones HTTP del usuario (o del frontend), valida los parámetros de entrada y delega la ejecución a la capa de servicios, retornando las respuestas adecuadas (códigos de estado HTTP).
- **service**: Encapsula la lógica de negocio central y orquesta las operaciones del sistema. Aquí se implementan los algoritmos de cálculo de morosidad, reglas de validación y la coordinación entre distintos repositorios o integraciones externas.
- **repository**: Abstrae la capa de acceso a datos (Persistencia). Contiene las interfaces que se comunican directamente con la base de datos relacional para realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre las entidades.
- **model (o domain)**: Contiene las clases conceptuales que representan el núcleo del negocio (**Usuario**, **Propiedad**, **Pago**, **GastoComun**), mapeadas directamente a las tablas de la base de datos (Entidades ORM).
- **dto (Data Transfer Object)**: Contiene objetos planos utilizados para transferir datos de forma

segura entre la capa *controller* y el cliente web, evitando exponer la estructura interna de la base de datos (**model**) y optimizando el payload de la red.

- **config**: Centraliza las configuraciones técnicas del sistema, tales como la inicialización de la base de datos, políticas de seguridad (CORS, JWT), y parámetros de entorno.
- **integration**: Agrupa los adaptadores y clientes HTTP encargados de la comunicación con APIs de terceros. Aquí residen las implementaciones concretas para la pasarela de pagos y el proveedor de servicios de correo electrónico (SMTP).
- **exception**: Gestiona el manejo centralizado de errores. Captura las excepciones lanzadas en cualquier capa del sistema y las formatea en respuestas estructuradas y comprensibles para el usuario o el cliente que consume la API.

4.3.2 Diagrama de Componente



4.3.3 Descripción Diagrama de Componentes

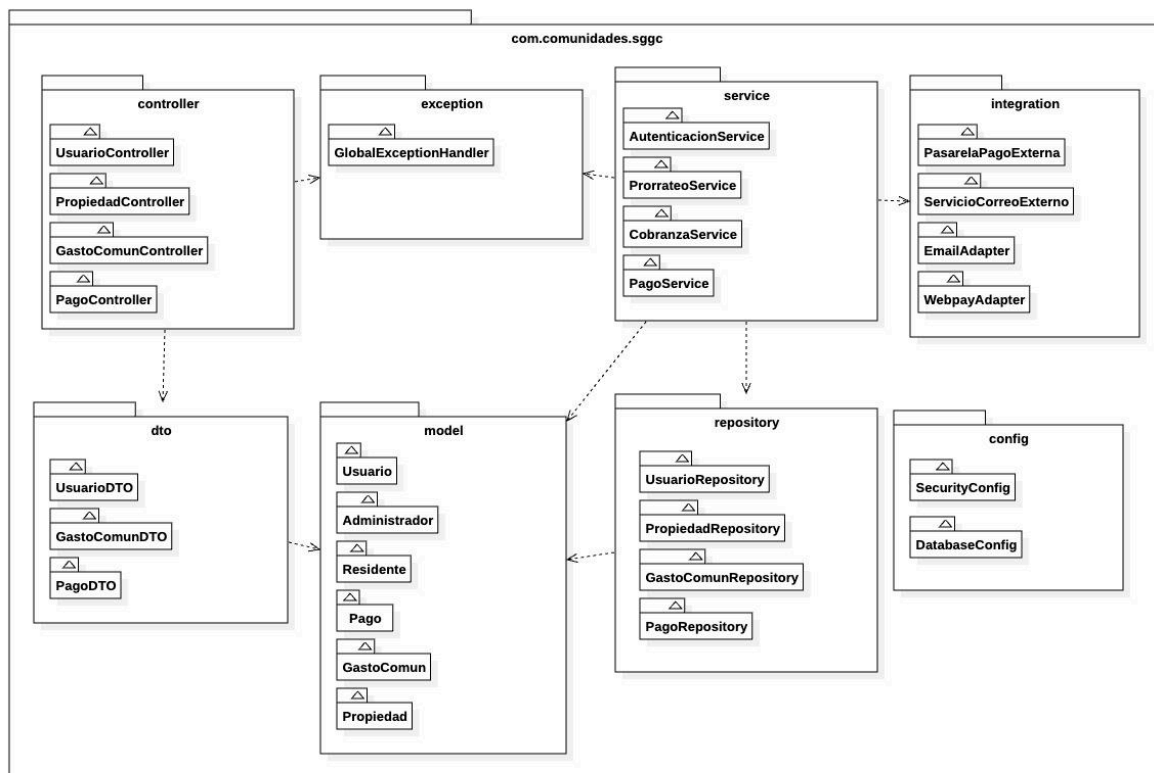
El diagrama de componentes representa la estructura modular y la organización física del Sistema de Gestión de Comunidades y Gastos Comunes, evidenciando el desacoplamiento alcanzado mediante un patrón de diseño arquitectónico basado en capas independientes conectadas a través de interfaces (*Ball-and-Socket*):

- **Web Frontend (Capa Cliente)**: Representa la aplicación web SPA con la cual interactúan los usuarios (Administradores y Residentes). Este componente consume los servicios expuestos por el backend mediante el requerimiento de la interfaz **API REST (HTTP/JSON)**.
- **Controller Component (Capa de Presentación Backend)**: Actúa como el punto de entrada controlado del servidor. Se encarga de exponer los endpoints, recibir las peticiones HTTP/JSON de la capa cliente, validar los parámetros de entrada y delegar de forma inmediata el flujo de control hacia la capa de negocio mediante el uso de la interfaz **Service**

API.

- **Service Component (Capa de Lógica de Negocio):** Es el núcleo inteligente de la aplicación. Centraliza todas las reglas del dominio del sistema, tales como la autenticación de usuarios, el procesamiento de las lógicas de las comunidades, la administración de propiedades, el cálculo automático de los algoritmos de prorrateo/morosidad para el cobro del **GastoComun**, y la orquestación de pagos.
- **Repository Component (Capa de Persistencia):** Abstrae y aísla por completo el acceso a los datos. Traduce los modelos conceptuales del negocio en operaciones de base de datos a través de un mapeo objeto-relacional (ORM), respondiendo a los requerimientos del servicio mediante la interfaz **Data Access API**.
- **Database (PostgreSQL):** El almacén físico relacional donde reside la información persistente del sistema, respondiendo a las peticiones del repositorio mediante la interfaz **SQL Connection**.
- **Integration Component (Capa de Infraestructura):** Componente especializado en aislar las comunicaciones con los proveedores de servicios externos de la lógica del negocio. Expone la interfaz **Integration API** hacia la capa de servicios y se conecta de forma asíncrona y desacoplada a las utilidades externas:
 - Establece una dependencia directa con la interfaz **PasarelaPagoExterna** para el procesamiento automatizado de Webpay.
 - Establece una dependencia directa con la interfaz **ServicioCorreoExterno** para interactuar con el servidor SMTP encargado del despacho de notificaciones, avisos de cobro y comprobantes.

4.3.4 Diagrama de paquete



4.3.5 Descripción Diagrama de Paquetes

El diagrama de paquetes describe la organización física y la estructura de directorios del código fuente bajo el espacio de nombres raíz del proyecto (**com.comunidades.sggc**). El modelo adopta las convenciones estandarizadas del framework Spring Boot, agrupando las clases e interfaces lógicas en paquetes modulares de alta cohesión, gobernados por una dirección de dependencia unidireccional que elimina por completo los acoplamientos cíclicos:

- **Paquete raíz (com.comunidades.sggc):** Actúa como el contenedor perimetral absoluto del sistema, encapsulando la totalidad de la lógica de la aplicación y asegurando el aislamiento del código frente a otros módulos de infraestructura.
- **Paquete controller (Capa de Entrada):** Contiene las clases conductoras de la API REST (**UsuarioController, PropiedadController, GastoComunController, y PagoController**). Su responsabilidad única es exponer los endpoints, interceptar los payloads JSON del frontend y delegar el procesamiento. Mantiene dependencias de uso directo (**<<use>>**) hacia los paquetes **service** y **dto**.
- **Paquete dto (Data Transfer Object):** Aloja las estructuras de datos planas encargadas del intercambio seguro de información con la capa cliente (**UsuarioDTO, GastoComunDTO, PagoDTO**). Depende exclusivamente de **model** para estructurar sus mapeos, evitando exponer de forma directa las entidades de persistencia hacia la red.
- **Paquete service (Capa de Negocio):** Es el componente de mayor densidad lógica y el núcleo del dominio. Aloja las clases **AutenticacionService, ProrratioService, CobranzaService y PagoService**. Coordina las transacciones críticas, tales como la ejecución de los algoritmos automáticos de prorratio y la validación de montos mayores a cero (**RN-08**). Depende de **model, repository, integration** y delega sus excepciones al paquete **exception**.
- **Paquete model (Dominio):** Contiene el núcleo conceptual de la arquitectura reflejado en las entidades ORM (**Usuario, Administrador, Residente, Propiedad, GastoComun, Pago**). Es el paquete con mayor nivel de independencia en el sistema, sirviendo como la base estructural sobre la cual operan los repositorios, los servicios y los DTOs.
- **Paquete repository (Capa de Persistencia):** Almacena las interfaces que heredan del ORM para la comunicación con la base de datos relacional (**UsuarioRepository, PropiedadRepository, GastoComunRepository, PagoRepository**). Su única dependencia se orienta de forma descendente hacia el paquete **model** para la ejecución de operaciones CRUD.
- **Paquete integration (Adaptadores de Infraestructura):** Centraliza el aislamiento de servicios de terceros para evitar el acoplamiento del negocio a proveedores específicos. Contiene las interfaces contractuales (**PasarelaPagoExterna, ServicioCorreoExterno**) y sus implementaciones o adaptadores concretos (**WebpayAdapter, EmailAdapter**), permitiendo la sustitución transparente de las plataformas tecnológicas de pago y mensajería.
- **Paquete config (Configuración Pasiva):** Aloja los componentes de soporte técnico y seguridad global (**SecurityConfig, JwtFilter, DatabaseConfig**). Se modela de manera aislada (sin flechas de dependencia salientes) debido al principio de Inversión de Control (IoC), siendo el propio framework el que inyecta sus directivas en tiempo de arranque.
- **Paquete exception (Manejo de Anomalías):** Centraliza la captura y el formateo de errores operacionales del sistema mediante componentes como **GlobalExceptionHandler, ResourceNotFoundException y BusinessException**. Recibe dependencias desde las capas de control y servicios, asegurando que ante fallos en la persistencia o violaciones a las reglas de negocio (como morosidades o montos inválidos) el servidor responda de forma

síncrona con códigos de estado HTTP estandarizados.

4.4. VISTA DE PROCESOS

4.4.1 Propósito

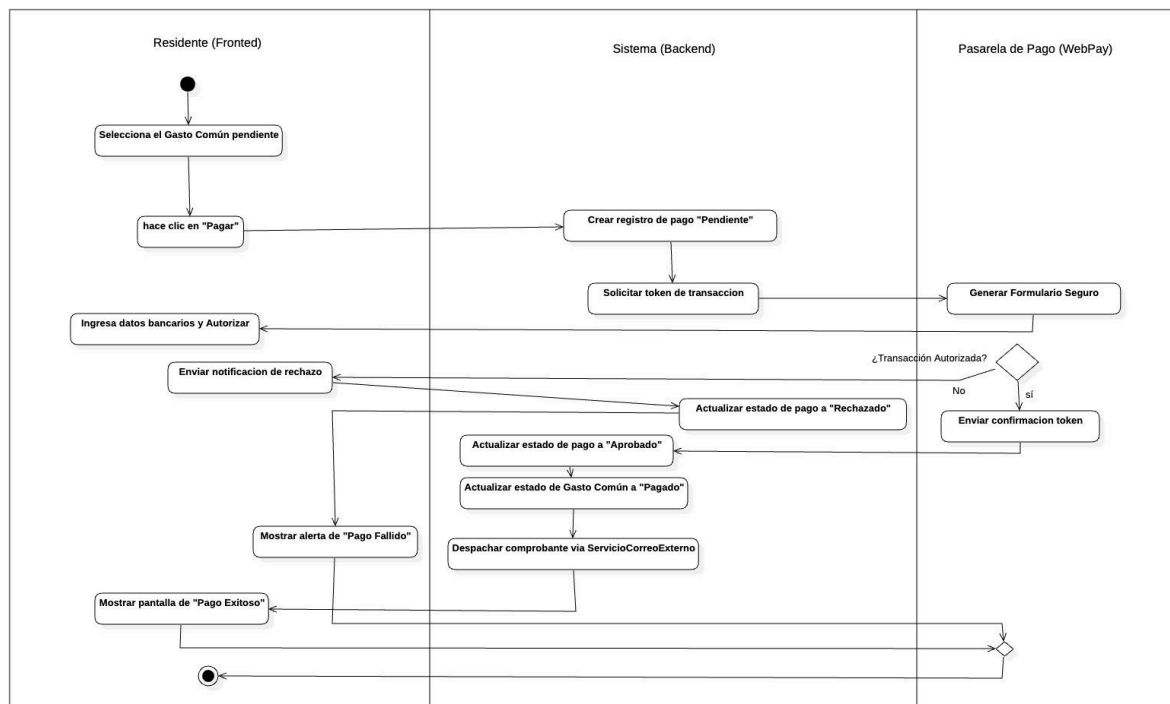
La Vista de Procesos describe el comportamiento dinámico del sistema y la interacción entre sus componentes durante la ejecución de una funcionalidad.

En el Sistema de Gestión de Comunidades y Gastos Comunes, esta vista permite visualizar cómo se ejecutan procesos relevantes, como el registro de pagos, validación de datos, comunicación con una pasarela de pago externa, actualización de saldos, generación de comprobantes y envío de notificaciones.

Esta vista es especialmente importante porque permite representar el flujo de actividades del sistema considerando tanto acciones internas como interacciones con servicios externos.

4.4.2 Diagrama de Actividad

Ejemplo :Registro de Pago con integración externa



4.4.3 Descripción del Diagrama de Actividad

El diagrama anterior detalla la secuencia lógica, asíncrona y transaccional requerida para completar el flujo de recaudación de un gasto común utilizando un proveedor financiero externo. El proceso distribuye las responsabilidades en tres carriles de ejecución (*Swimlanes*):

- **Residente (Frontend):** Representa la capa de interacción del usuario. El flujo inicia cuando

este selecciona un gasto común pendiente en la interfaz y hace clic en "Pagar". Más adelante, el carril reanuda su actividad al recibir el formulario bancario seguro para ingresar sus datos y autorizar el cobro. Finalmente, el frontend intercepta la respuesta del servidor para renderizar visualmente una alerta de "Pago Fallido" o una pantalla de "Pago Exitoso" con su respectivo comprobante.

- **Sistema (Backend):** Actúa como el orquestador y validador de las reglas de negocio del dominio. Al recibir la solicitud inicial, genera inmediatamente un registro en la base de datos en estado "**Pendiente**" y solicita el token transaccional seguro a la API de pagos. Si la pasarela rechaza la operación, el backend muta el estado a "**Rechazado**". Si es aprobada, ejecuta las actualizaciones críticas en cadena: cambia el estado del pago a "**Aprobado**", marca el gasto común como "**Pagado**", rebaja los saldos correspondientes de la **Propiedad (RN-08)** y gatilla el despacho del comprobante digital a través de la interfaz **ServicioCorreoExterno**.
- **Pasarela de Pago (WebPay):** Componente perimetral externo encargado de la seguridad bancaria. Recibe la orden del backend, genera el formulario seguro para el cliente, procesa la transacción con la entidad financiera del residente y evalúa la condición mediante un nodo de decisión (*¿Transacción Autorizada?*). Dependiendo del resultado, despacha una notificación de rechazo o envía la confirmación con el token firmado hacia el backend para consolidar la operación.

4.5. VISTA FÍSICA

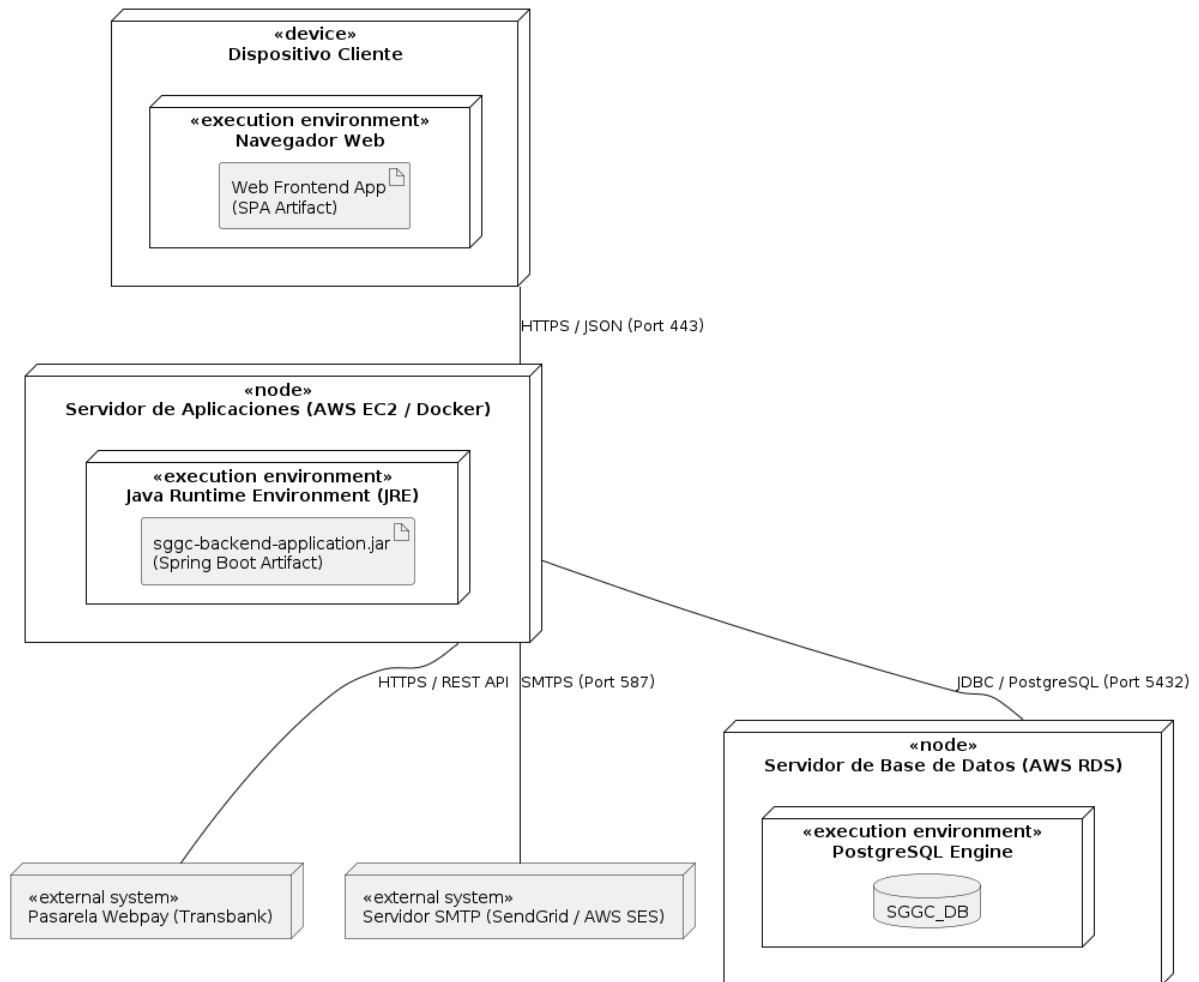
4.5.1 Propósito

La Vista Física describe cómo se distribuye el sistema en la infraestructura tecnológica donde será ejecutado. Su objetivo es representar los nodos físicos o virtuales que participan en el funcionamiento del sistema, tales como dispositivos cliente, servidor de aplicación, base de datos y servicios externos.

En este sistema, la Vista Física también representa la comunicación con servicios externos de pago y correo electrónico, los cuales son necesarios para procesar transacciones y enviar notificaciones automáticas.

4.5.2 Diagrama de despliegue

Diagrama de Despliegue - Sistema de Gestión de Comunidades



4.5.3 Descripción Diagrama de Despliegue

El diagrama de despliegue detalla la topología de red y la distribución física de la infraestructura tecnológica sobre la cual opera el Sistema de Gestión de Comunidades y Gastos Comunes. El diseño adopta un enfoque descentralizado y basado en servicios en la nube, garantizando el aislamiento de las capas para mejorar la escalabilidad, la seguridad y el rendimiento:

- **Nodo «device» Dispositivo Cliente:** Representa el hardware físico (computadores de escritorio, notebooks, smartphones o tablets) utilizado por los Residentes y Administradores para acceder a la plataforma.
 - **Entorno de Ejecución («execution environment» Navegador Web):** Entorno de software del cliente (como Google Chrome, Mozilla Firefox o Safari) que hospeda e interpreta el artefacto **Web Frontend App (SPA)**.
 - **Protocolo de Comunicación:** Se comunica de forma exclusiva con el servidor de aplicaciones mediante solicitudes asíncronas bajo el protocolo seguro **HTTPS** (Puerto standard 443), intercambiando cargas de datos en formato estandarizado **JSON**.
- **Nodo «node» Servidor de Aplicaciones (AWS EC2 / Contenedor Docker):** Instancia de

cómputo virtualizada en la nube que aloja el entorno del backend de la plataforma.

- **Entorno de Ejecución («execution environment» JRE):** Entorno de ejecución de Java (Java Runtime Environment) necesario para levantar el artefacto de despliegue empaquetado **sggc-backend-application.jar** generado por el framework Spring Boot.
- **Responsabilidad:** Procesa las peticiones REST entrantes, intercepta los tokens JWT mediante sus filtros de seguridad, ejecuta las lógicas críticas de la comunidad (cálculo de prorrates y asignación de morosidades) y orquesta la comunicación con las dependencias perimetrales.
- **Nodo «node» Servidor de Base de Datos (AWS RDS):** Servidor dedicado y aislado físicamente del servidor de aplicaciones para maximizar la seguridad de los registros financieros y personales.
 - **Entorno de Ejecución («execution environment» PostgreSQL Engine):** Motor de base de datos relacional encargado de administrar de forma óptima el almacenamiento físico de la base de datos persistente del sistema.
 - **Protocolo de Comunicación:** El servidor de aplicaciones se conecta de forma directa a este nodo mediante el driver nativo **JDBC / PostgreSQL** viajando sobre una conexión segura TCP/IP a través del puerto estándar **5432**.
- **Nodo «external system» Pasarela Webpay (Transbank):** Infraestructura externa y descentralizada perteneciente al proveedor transaccional. El servidor de aplicaciones del backend establece un canal de comunicación asíncrono con este nodo mediante peticiones cifradas **HTTPS / REST API** para iniciar transacciones financieras seguras, validar tokens de pago y recibir confirmaciones de transacciones de forma segura y desacoplada.
- **Nodo «external system» Servidor SMTP (SendGrid / AWS SES):** Plataforma perimetral en la nube de un tercero dedicada exclusivamente al despacho masivo y asíncrono de mensajería electrónica. El backend consume este servicio mediante el protocolo seguro institucional **SMTPS** (usualmente sobre el puerto de red **587** o **465**) para externalizar el envío de los avisos de cobranza por propiedad, alertas automáticas de morosidad y los comprobantes digitales de pago visados.

5. REQUISITOS DE CALIDAD

5.1 Propósito

El propósito de los requisitos de calidad es definir las características no funcionales que deberá cumplir el sistema para asegurar un funcionamiento eficiente, seguro, accesible y fácil de mantener.

Estos requisitos permiten garantizar una buena experiencia para los usuarios, facilitar el trabajo del equipo de desarrollo y asegurar que el sistema pueda evolucionar en el tiempo sin afectar su rendimiento o estabilidad.

Además, los requisitos de calidad ayudan a establecer criterios de evaluación para verificar que la solución desarrollada cumple con los estándares esperados de usabilidad, accesibilidad, rendimiento y seguridad del software.

5.2 Atributos de calidad

ATRIBUTO DE CALIDAD	DESCRIPCIÓN	JUSTIFICACIÓN
Usabilidad	El sistema debe ser intuitivo, fácil de aprender y sencillo de utilizar para los usuarios. Debe contar con interfaces claras, mensajes comprensibles y navegación simple.	Permite mejorar la experiencia de usuario y reducir errores durante el uso del sistema. Facilita la adopción de la plataforma por parte de usuarios con distintos niveles de conocimiento tecnológico.
Accesibilidad (WCAG)	El sistema deberá considerar principios de accesibilidad como perceptibilidad, operabilidad, comprensibilidad y robustez, permitiendo el acceso a personas con distintas capacidades.	Garantiza inclusión digital y cumplimiento de estándares de accesibilidad web, permitiendo que más usuarios puedan utilizar la plataforma correctamente.
Rendimiento	El sistema deberá responder de manera rápida y eficiente ante solicitudes de usuarios, minimizando tiempos de carga y procesamiento.	Un buen rendimiento mejora la satisfacción del usuario y evita retrasos en procesos importantes del sistema.
Mantenibilidad	El software deberá desarrollarse utilizando una arquitectura modular y organizada, facilitando correcciones, mejoras y futuras actualizaciones.	Permite reducir costos de mantenimiento y facilita el trabajo del equipo de desarrollo durante la evolución del sistema.
Seguridad	El sistema deberá proteger la información mediante autenticación, validación de datos y control de acceso a funcionalidades.	La seguridad es fundamental para proteger datos sensibles y evitar accesos no autorizados o pérdida de información.
Portabilidad	El sistema deberá poder ejecutarse en distintos dispositivos y navegadores, manteniendo compatibilidad y correcto funcionamiento.	Permite que los usuarios accedan desde computadores, tablet o teléfonos móviles sin afectar la experiencia de uso.
Escalabilidad	La arquitectura deberá permitir agregar nuevas funcionalidades o aumentar la cantidad de usuarios sin afectar el funcionamiento general.	Facilita el crecimiento futuro del sistema y la incorporación de nuevos módulos o servicios.
Disponibilidad	El sistema deberá mantenerse operativo la mayor parte del tiempo, minimizando interrupciones o caídas del servicio.	Asegura continuidad operacional y confianza de los usuarios en la plataforma.

5.3. Reglas y criterios de evaluación de calidad.

Para garantizar el cumplimiento de atributos de calidad definidos anteriormente se establecen las

siguientes reglas y criterios de evaluación:

- Las interfaces del sistema deberán mantener consistencia visual y navegación clara en todas sus pantallas.
- El sistema deberá validar datos ingresados por el usuario y mostrar mensajes de error comprensibles.
- El tiempo de respuesta de las operaciones principales no deberá superar los 3 segundos en condiciones normales de uso.
- La aplicación deberá ser compatible con navegadores modernos como Google Chrome, Microsoft Edge y Mozilla Firefox.
- El sistema deberá adaptarse correctamente a dispositivos móviles y diferentes resoluciones de pantalla.
- Todas las funcionalidades deberán contar con mecanismos de control de acceso según el tipo de usuario.
- El código deberá mantener separación por capas y modularidad para facilitar mantenimiento y escalabilidad,
- Las pruebas de funcionamiento deberán verificar tanto requisitos funcionales como atributos de calidad.
- Se aplicarán principios de accesibilidad WCAG para mejorar la interacción de usuarios con distintas capacidades.
- El sistema deberá permitir futuras integraciones con servicios externos mediante APIs o microservicios.

Basándose en estos criterios, el sistema podrá ser evaluado para determinar si cumple con los estándares de calidad definidos para el proyecto.

5. BIBLIOGRAFÍA

- Arquitectura de Software y UML:
 - Fowler, M. (2018). *Refactorización: Mejora del diseño del código existente* (2da ed.). Addison-Wesley Professional.
 - Object Management Group (OMG). (2015). *Especificación del Lenguaje Unificado de Modelado (UML) v2.5.1*. <https://www.omg.org/spec/UML/2.5.1/About-UML/>
 - Sommerville, I. (2016). *Ingeniería de software* (10a ed.). Pearson Educación.
- Tecnologías del Backend (Spring Boot y Java):
 - Oracle Corporation. (2025). *Documentación de Java Platform, Standard Edition (Java SE) 21*. <https://docs.oracle.com/en/java/javase/21/>
 - Comunidad de Spring Boot. (2026). *Documentación de Referencia de Spring Boot (v3.4.x)*. VMware Tanzu. <https://docs.spring.io/spring-boot/index.html>
 - Walls, C. (2022). *Spring en Acción* (6ta ed.). Manning Publications.
- Persistencia y Bases de Datos (PostgreSQL y ORM):
 - Bauer, C., King, G., y Metz, G. (2015). *Persistencia en Java con Hibernate* (2da ed.). Manning Publications.
 - Grupo de Desarrollo Global de PostgreSQL. (2026). *Documentación Oficial de PostgreSQL 16.0*. <https://www.postgresql.org/docs/16/>
- Integraciones y Servicios de Terceros (Webpay y Mensajería):
 - Transbank Developer. (2025). *Documentación Oficial de Integración API Webpay Plus*. Transbank S.A. <https://www.transbankdevelopers.cl/referencia/webpay>
 - Amazon Web Services. (2026). *Guía del Desarrollador de Amazon Simple Email Service (SES)*. Documentación de AWS. <https://docs.aws.amazon.com/ses/>
- Herramientas de Modelado:
 - MKLab. (2024). *Manual de Usuario de StarUML 6.0*. <https://staruml.io/docs>

